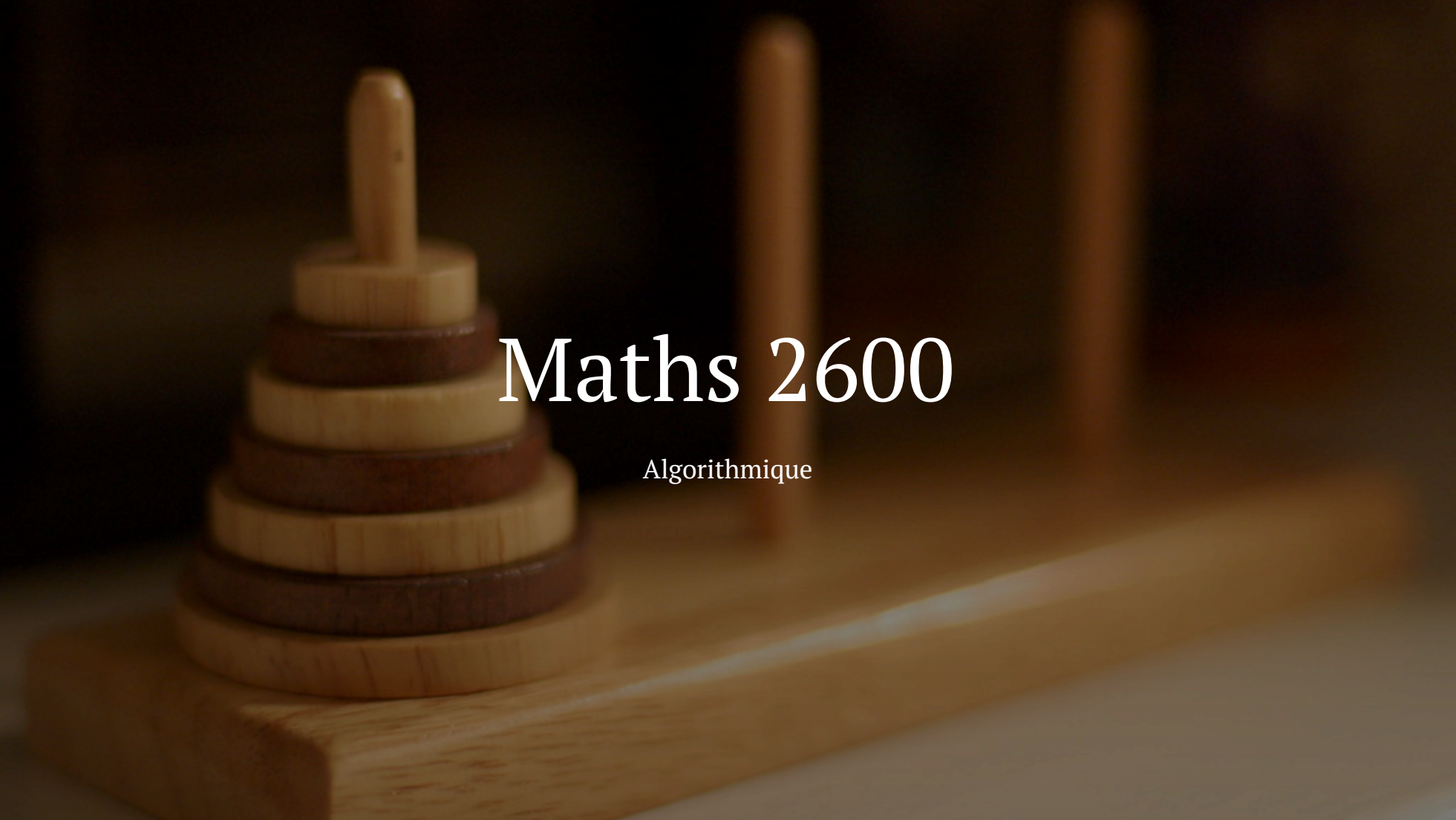


A wooden Tower of Hanoi puzzle is shown in a close-up, slightly blurred shot. The puzzle consists of three vertical wooden pegs and several horizontal wooden rings of varying sizes. One of the rings is currently stacked on the leftmost peg, while the others are stacked on the middle and right pegs. The background is dark and out of focus.

Maths 2600

Algorithmique

Récapitulatif

Ce qu'on a vu

- Fondations des mathématiques
- Formalisation
- Calculabilité
 - Machines de Turing comme formalisation mathématique du calcul
 - Limites théoriques du calcul
 - Complexité

Ce qu'on va voir

- Calculer dans un cadre plus pratique que les machines de Turing
- Analyse de problèmes et écriture d'algorithmes en pseudo-code
- Calcul de complexité
- Structures de données

Ecrire un algorithme

Le problème à résoudre

- On part d'un problème à résoudre de manière algorithmique pour pouvoir l'implémenter en code
- Un problème à des entrées et des sorties
- Si on écrit une fonction pour le résoudre, les entrées sont les paramètres de la fonction, et les sorties ce qui sera renvoyé

Exemples de problèmes

- Problème de base, très "algo":
 - Calcul d'une fonction numérique
 - Tri d'une liste
- Problème plus compliqué, mais très "algo" aussi:
 - Compilation d'un code source
- Problème très applicatif:
 - Calcul d'un plan de batch cooking: une liste de recette optimale à préparer le week end pour être distribué la semaine
- Problème de thunes:
 - Calcul de points d'achat / vente pour un bot de trading
- Problèmes mathématiques / physiques:
 - Rendu 3D: calcul de la projection d'une scène sur un plan virtuel représentant une caméra
 - Simulation des interactions de particules dans le temps au sein d'un liquide

Trouver un algorithme

- L'ensemble de tous les problèmes est très varié !
 - Il n'y a pas de formule "générique" pour écrire un algorithme, le contexte est important
 - Mais il y des astuces
- Se munir d'une feuille et d'un crayon (eventuellement une tablette et un soft de dessin)
- Tenter d'explorer l'espace du problème à l'écrit
 - Se donner des exemples d'entrée simple
 - Tenter de résoudre le problème à la main sur ces exemples simples
 - En extraire la procédure de résolution
 - Ecrire cette procédure en pseudo-code

Problèmes avancés

- Le problème de simulation des interactions de particules, bon courage pour l'explorer à la main
- Mais ce genre de problème est déjà très avancé
 - En général on maîtrise déjà bien la programmation avant de s'y attaquer
 - Sur ce type de problème on tentera plutôt d'explorer le cadre théorique lié au problème
 - Par exemple en rendu 3D, on étudiera la géométrie dans l'espace, les modèles de matériaux, la simulation de lumière, ...
 - En trading algorithmique on étudiera les indicateurs techniques, les stratégies de trading, les statistiques et probabilité, la gestion du risque, ...
 - En cryptographie on étudiera la théorie des nombres, les statistiques, la théorie des courbes elliptiques
- De manière générale, on étudie le domaine d'application avant de s'attaquer aux algorithmes d'un champs précis et on cherche à caractériser précisément la structure des éléments qui le compose
 - Sur une application de recettes, on définira précisément ce qu'est une recette, ce qu'est un ingrédient et les relations entre ces éléments
 - C'est de la modélisation et de la formalisation

Algorithmes numériques

Exploration d'un algorithme numérique

- On doit implémenter une fonction qui manipule des nombres
- En général l'exploration sur papier est faisable

Exemple: Le retour de Fibonacci

- On cherche à calculer la suite de Fibo:
 - $F_0 = 0$
 - $F_1 = 1$
 - $F_n = F_{n-2} + F_{n-1}$ pour $n \geq 2$
- On explore le problème: une suite de nombres entiers
 - Suite de nombre entier = une fonction qui prend un nombre entier et qui renvoie un nombre entier
 - `def fibo(n: int) -> int`
 - On liste des valeurs de la suite et on regarde comment on les calcule à la main:
 - $F_0 = \text{fibo}(0) \Rightarrow 0, F_1 = \text{fibo}(1) \Rightarrow 1$
 - $\text{fibo}(2) = F_2$? On remplace n par 2 dans la formule $\Rightarrow F_{2-2} + F_{2-1} = F_0 + F_1 = 0 + 1 = 1$
 - $\text{fibo}(3) = F_3$? On remplace n par 3 dans la formule $\Rightarrow F_{3-2} + F_{3-1} = F_1 + F_2 = 1 + 1 = 2$
 - $\text{fibo}(4) = F_4$? On remplace n par 4 dans la formule $\Rightarrow F_{4-2} + F_{4-1} = F_2 + F_3 = 1 + 2 = 3$
 - $\text{fibo}(5) = F_5$? On remplace n par 5 dans la formule $\Rightarrow F_{5-2} + F_{5-1} = F_3 + F_4 = 2 + 3 = 5$

Exemple: Le retour de Fibonacci

$$F_n = F_{n-2} + F_{n-1}$$

- On identifie des propriétés du problème et de l'algorithme:
 - a chaque étape j'utilise des valeurs précédentes pour faire mon calcul
 - => construire une table ?
 - a chaque étape j'utilise uniquement 2 valeurs précédentes pour faire mon calcul
 - => besoin que de deux variables, pas d'une table
 - pour calculer jusqu'à une certaine valeur, j'ai besoin de toutes celles d'avant
 - => je dois boucler
- On exprime un début l'algo en pseudo-code:

```
fonction fibo(n: entier) -> entier:
    # je déclare mes deux variables précédentes
    f_n_2 = 0
    f_n_1 = 1
    de i = 1 à n:
        # je pose un début de boucle; chaque étape doit traduire une ligne de mon calcul fait à la main
        f_n = f_n_2 + f_n_1
    renvoyer f_n
```

Exemple: Le retour de Fibonacci

```
fonction fibo(n: entier) -> entier:
  # je déclare mes deux variables précédentes
  f_n_2 = 0
  f_n_1 = 1
  de i = 1 à n:
    # je pose un début de boucle; chaque étape doit traduire une ligne de mon calcul fait à la main
    f_n = f_n_2 + f_n_1
  renvoyer f_n
```

- Je réfléchis si mon algo fonctionne en l'essayant, je le raffine
 - `fibo(0)` => je ne rentre pas dans la boucle, `f_n` n'est pas défini
 - `fibo(1)` => je rentre dans la boucle et je calcule `f_n_2 + f_n_1`, donc $F_{n-2} + F_{n-1} = F_{1-2} + F_{1-1} = F_{-1} + F_0$ => il y a un souci conceptuel.
 - `fibo(2)` => je rentre dans la boucle et je calcule `f_n_2 + f_n_1` deux fois (deux tour de boucle). Mais `f_n_2` et `f_n_1` restent les mêmes donc j'obtiens toujours `0 + 1`

Exemple: Le retour de Fibonacci

```
fonction fibo(n: entier) -> entier:
    f_n_2 = 0
    f_n_1 = 1
    # on gère séparément les cas n = 0 et n = 1
    si n == 0:
        renvoie f_n_2
    si n == 1:
        renvoie f_n_1

    de i = 2 à n:
        # maintenant on sait qu'on a minimum n = 2, donc i = 2 au départ
        f_n = f_n_2 + f_n_1
        # il faut faire évoluer f_n_2 et f_n_1 pour être sûr d'avoir toujours les deux précédents
        f_n_2 = f_n_1
        f_n_1 = f_n

    renvoyer f_n
```

Exemple: Le retour de Fibonacci

- On optimise

```
fonction fibo(n: entier) -> entier:
  f_n_2 = 0      # F_0
  f_n_1 = 1      # F_1
  de i = 1 à n:
    f_n = f_n_2 + f_n_1  # au premier tour de boucle F_0 + F_1 => donc F_2
    f_n_2 = f_n_1        # au deuxieme tour de boucle f_n_1 vaut F_2, donc f_n_2 vaudra F_2
    f_n_1 = f_n

  renvoyer f_n_2
```


Exemple: Le retour de Fibonacci

- On renomme les variable pour eviter de la confusion

```
fonction fibo(n: entier) -> entier:
  f_n = 0          # on le renomme f_n car c'est celui qu'on renvoie, donc correspond fibo(n)
  f_n_plus_1 = 1
  de i = 1 à n:
    f_n_plus_2 = f_n + f_n_plus_1
    f_n = f_n_plus_1
    f_n_plus_1 = f_n_plus_2

  renvoyer f_n
```

- Correspond à la definition alternative $F_{n+2} = F_n + F_{n+1}$ équivalente à $F_n = F_{n-2} + F_n$

Pseudo-code et Test Driven Development

- A la place du pseudo-code on peut faire l'implémentation en TDD
- Mais il faut quand même l'exploration papier !
 - Permet d'identifier des cas de test à implémenter
 - On écrit les tests
 - On implémente la fonction itérativement jusqu'à passer les tests
- Avantage: un retour immédiat
- Inconvenient
 - Si on va trop vite, peut cacher une partie de la compréhension du problème
 - On peut facilement passer à côté de certains cas si on coupe l'exploration trop tot

Exemple: Le retour de l'algorithme de l'addition

- On veut additionner des nombres représenté par leur chaine de caractère en base 10

- Une fonction `add(a: string, b: string) -> string`

- On se donne quelques exemples:

- `add("42", "124") == "166"`
 - `add("1024", "5") == "1029"`
 - `add("36", "95") == "131"`
 - `add("0", "48") == "48"`
 - `add("28", "0") == "28"`

Handwritten addition examples illustrating the algorithm:

Example 1:
$$\begin{array}{r} 124 \\ + 42 \\ \hline 166 \end{array}$$

Example 2:
$$\begin{array}{r} 036 \\ + 95 \\ \hline 131 \end{array}$$

- On calcule les exemple sur papier avec l'algorithme pour comprendre comment l'implémenter

Exemple: Le retour de l'algorithme de l'addition

- On identifie des propriétés du problème et de l'algorithme:
 - On parcourt les symboles de droite à gauche
 - Parcourt = une boucle sur les caractères des string
 - De droite à gauche = boucle inversée
 - Ou alors on peut inverser les chaines pour se simplifier la vie
 - On doit additionner les symboles individuellement => on veut une table d'addition sur les symboles
 - En C on peut faire ça via de l'arithmétique sur le code ASCII
 - On doit gérer une retenue
 - Il faut savoir si une addition de symbole donne une retenue
 - Il faut la stocker
 - La retenue au toujours 0 ou 1
 - On met toujours le plus grand nombre en haut

Exemple: Le retour de l'algorithme de l'addition

```
fonction add(a: string, b: string) -> string:
    si longueur(b) > longueur(a):
        a, b = b, a
    a = reverse(a)
    b = reverse(b)

    b = completer_avec_des_zero(b, longueur(a))

    output = ""
    retenue = 0

    de i = 0 à longueur(b) - 1:
        resultat, retenue = table_addition(a[i], retenue, b[i])
        output += resultat

    si retenue:
        output += "1"

    return output
```

Algorithmes sur les listes

Exemple: Recherche dans une liste

- On a une liste et un element, on veut renvoyer la position de l'element dans cette liste ou -1 s'il n'y est pas
 - `fonction recherche(l: liste, x: any) -> int`
- On explore le problème sur papier

Exemple: Recherche dans une liste

- On identifie des propriétés du problème et de l'algorithme:
 - On parcourt la liste de gauche à droite => une boucle

```
fonction recherche(l: liste, x: any) -> int:  
  de i = 0 à longueur(l) - 1:  
    si l[i] == x:  
      renvoyer i  
  renvoyer -1
```


Exemple: Recherche dans une liste triée

- On a une liste triée et un element, on veut renvoyer la position de l'element dans cette liste ou -1 s'il n'y est pas
 - `fonction recherche_triée(l: liste, x: any) -> int`
- On explore le problème sur papier

Exemple: Recherche dans une liste

- On identifie des propriétés du problème et de l'algorithme:
 - Il semble qu'on puisse faire mieux que tout parcourir
 - On peut faire ce qu'on appelle de la dichotomie

```
fonction recherche_triee(l: liste, x: any) -> int:  
    debut = 0  
    fin = longueur(l) - 1  
    faire:  
        milieu = (debut + fin) / 2  
        si x == l[milieu]:  
            renvoyer milieu  
        si x < l[milieu]:  
            fin = milieu  
        si x > l[milieu]:  
            debut = milieu  
    tant que debut != milieu  
  
    renvoyer -1
```

Exemple: Tri d'une liste

- On a une liste qu'on veut trier
 - `fonction trier(l: liste) -> liste`
- On explore le problème sur papier

Exemple: Tri d'une liste

- On identifie des propriétés du problème et de l'algorithme:
 - A chaque étape, on cherche le plus petit element
 - On le met au début
 - Il faut trouver ou mette l'element du début
 - On peut swapper les elements

Exemple: Tri d'une liste

```
fonction trier(l: liste) -> liste:  
  l = copy(l)  
  de j = 0 à longueur(l):  
    min_index = j  
    de i = j à longueur(l):  
      si l[i] < l[min_index]:  
        min_index = i  
    l[j], l[min_index] = l[min_index], l[j]  
  renvoyer l
```

Calcul de complexité

Objectif

- Etant donné un pseudo-code, on cherche à évaluer sa complexité en temps
- Sa complexité = formule exprimant de manière asymptotique le nombre d'étape du calcul en fonction de la taille de l'entrée
- Peut être compliqué à calculer sur des algorithmes complexes, mais on a quelques règles qui nous aide

Structures de base du langage

- Etant donné un pseudo-code, on va additionner une estimation du coup de calcul de ses structures de base
- Une instruction de base sur des objets de taille bornée (int32, float) s'exécute en temps constant $O(1)$
 - Opérateurs, affectation de variable, comparaison, etc
- Dans le cas d'un `if` , `else` , on prendra le max des branches qui le constitue
 - Dans certains cas il est possible d'être plus fin, mais en première approche c'est bien
- Dans le cas d'une boucle, on multipliera le coût de son corps par le nombre de tours de boucle
- Si on appelle une autre fonction, on devra calculer sa complexité pour l'intégrer dans la formule de complexité de la fonction appelante
- Pour obtenir le comportement asymptotique, on retirera les constantes multiplicatrices et additives

Exemple: Recherche dans une liste

```
fonction recherche(l: liste, x: any) -> int:  
  de i = 0 à longueur(l) - 1:  
    si l[i] == x:  
      renvoyer i  
  renvoyer -1
```

- On choisit $n = \text{longueur}(l)$
- Boucle de n tours: on multiplie n par le coût du corps de la boucle
- Le corps:
 - Une comparaison, temps constant
 - Renvoyer, temps constant
- Donc le coût global est de l'ordre de $n \times C$ ou C est une constante
- On dit que l'algorithme est en temps linéaire noté $O(n)$

Exemple: Recherche dans une liste triée

```
fonction recherche_triée(l: liste, x: any) -> int:  
    debut = 0  
    fin = longueur(l) - 1  
    faire:  
        milieu = (debut + fin) / 2  
        si x == l[milieu]:  
            renvoyer milieu  
        si x < l[milieu]:  
            fin = milieu  
        si x > l[milieu]:  
            début = milieu  
    tant que debut != milieu  
  
    renvoyer -1
```

- Une boucle, mais condition compliqué pour trouver le nombre de tour
 - A chaque étape, on coupe en deux (divise par 2)
 - Itération de division = inverse d'itération de multiplication, qui est la puissance
 - Inverse de 2^n c'est $\log_2(n)$
 - Corps de la boucle en temps constant, donc l'algorithme s'exécute en temps logarithmique $O(\log_2(n))$

Calcul de complexité dans le cas récursif

- Pour une implémentation recursive, la fonction de coût qu'on va calculer sera également récursive
- On cherchera alors à la réduire à une formule simple, ce qui peut généralement être fait de manière intuitive
- Quand c'est plus compliqué, on doit utiliser des maths assez avancer comme le calcul de série entières

Exemple: Recherche dans une liste triée en récursif

```
fonction recherche_triée(l: liste, x: any, index = 0) -> int:
    si longueur(l) == 0:
        renvoyer -1

    si longueur(l) == 1:
        renvoyer index si l[0] == x sinon -1

    milieu = (longueur(l) - 1) / 2
    si x < l[milieu]:
        renvoyer recherche_triée(l[0:milieu], x, index)

    renvoyer recherche_triée(l[milieu:], x, index + milieu)
```

- La fonction de cout aura la forme récursive suivante: $cout(n) = C + cout(n/2)$
 - Si on déroule le comportement de cette fonction on à:
 - $cout(n) = C + C + cout(n/4) = C + C + C + cout(n/8)$
 - A un moment la liste est vide ou a un seul element, le cout devient constant uniquement
 - On a donc une addition de constantes contenant $\log_2(n)$ termes
 - => complexité $O(\log_2(n))$

Optimisation d'implémentation

- Optimiser une implémentation, c'est chercher un algorithme de complexité plus faible pour résoudre le même problème
- Attention néanmoins:
 - La théorie de la complexité s'intéresse au comportement asymptotique, pour de grandes entrées
 - Si on a jamais à traiter de grandes entrées, il sera parfois plus efficace de choisir un algorithme de moins bonne complexité mais qui se comporte mieux sur de petites entrées
 - En général, il faut bien connaître les données qu'on traite pour trouver la meilleure implémentation

Exemple: Tri fusion d'une liste

Le cas des algorithmes numériques

- Ce qui nous interesse dans le cas du calcul de complexité d'un algorithme numérique, c'est la taille mémoire de l'entrée, pas sa valeur
- La taille d'une entrée de type entier, c'est son nombre de bits
 - Si nos entiers sont sur 32-bits, c'est une taille constante, donc en théorie la fonction qu'on implémente aura toujours un nombre d'étapes borné par une constante (potentiellement très grande), sa complexité est donc $O(1)$
 - Si nos entiers sont de taille arbitraire, alors le nombre de bit est de l'ordre de $\log_2(k)$ ou k est le nombre en entrée
 - Si on pose $n = \log_2(k)$ le nombre de bits, on a donc $k = 2^n$
 - Donc si j'ai une boucle qui va de 0 à k , j'ai en réalité un coût exponentiel
 - Pour un coût linéaire, je dois m'arranger pour faire des boucles sur le nombre de bits
- Remarque: en python on a des entiers de taille arbitraire, les opérations de base peuvent donc être considéré comme $O(\log_2(k))$ sur ces nombres
 - En général on considère $O(1)$ sauf si ça a du sens de prendre le nombre de bits en compte (crypto)

Exemple: Somme des entiers de 0 à k en temps exponentiel

Exemple: Somme des entiers de 0 à k en temps linéaire

Structures de Données

Pourquoi les structures de données ?

- Une structure de donnée est une manière de modéliser des stocker des données. Utiles pour:
 - Représenter les données d'un problème de manière adaptée à son traitement
 - Stocker les données d'une application d'une manière adaptée aux algorithmes qu'on y appliquera
- Les structures de données de base peuvent être combinées pour produire des structures de données plus complexes
- Structures de base:
 - Structures séquentielles: tableaux et listes
 - Structures hierarchiques: arbres, graphes
 - Structures d'indexage: tables de hachage
- Les structures de données peuvent être formalisées dans un langage mathématique
 - En donnant un ensemble décrivant ce type de structure
 - En donnant des propriétés sur cet ensemble
 - En donnant des opérations
 - En donnant des axiomes sur ces opérations

Doit t-on implémenter les structures de données de base ?

- A part dans les langages bas niveau, c'est assez rare
 - Soit présent dans la bibliothèque standard (C++, Java)
 - Soit présent en temps que type primitif du langage (python)
- Ca reste un très bon exercice qu'on fait généralement en C
 - Permet de bien comprendre les algorithmes de bases et l'impact du choix d'une structure sur la complexité
 - Permet de bien comprendre les pointeurs et la mémoire

Structures séquentielles

- Dans les langages très au niveau en général on assimile liste et tableau
 - Une liste est une structure indexable, on peut ajouter au début ou à la fin, obtenir sa taille, etc
- Sur un langage plus bas niveau, on distingue les tableau des listes dites chaînées
 - Un tableau représente un espace continu en mémoire
 - Il est efficace de l'indexer et calculer sa taille
 - Par contre ajouter un element, ou les réordonner peut être long
 - Une liste chaînée représente des elements dispatchés en mémoire, relié entre eux via des pointeurs ou références
 - Il est efficace d'ajouter ou réordonner les éléments
 - Par contre indexer ou calculer sa taille est long

Structures séquentielles de plus haut niveau

- On formalise des structures plus haut niveau, possédant des opérations avec une sémantique précise
- La file (liste FIFO)
 - Représente une file d'attente
 - On peut ajouter a la fin, retirer au début
- La pile (list LIFO)
 - Représente une pile d'assiette
 - On peut ajouter au début, retirer au début
- Le fait de formaliser ces concepts permet de s'y referer dans certaines algorithmes

Arbres

- Les arbres représentent des structures hiérarchiques
 - Un arbre est constitué de nœuds
 - Chaque nœud a des nœuds enfants, souvent liés par référence (pointeurs en C)
 - Les nœuds sans enfants sont les feuilles
 - Les nœuds sans parents sont les racines
- On parle d'arbre binaire quand tous les nœuds ont 0, 1 ou 2 enfants
 - Souvent utilisés pour implémenter des tris, ou recherche efficaces
 - Ou bien des arbres sémantiques représentant des formules d'opérations binaires
- Les algorithmes typiques sur les arbres sont les parcours
 - En profondeur
 - En largeur

Exemple: Algorithme de parcours en profondeur (Recuratif)

```
function parcours_profondeur_rec(noeud: Noeud):  
    do_something_with_noeud(noeud)  
  
    pour chaque fils de enfants(noeud):  
        parcours_profondeur(fils)
```


Exemple: Algorithme de parcours en profondeur (Iteratif)

```
function parcours_profondeur_it(noeud: Noeud):  
    pile: LIFO = [ noeud ]  
  
    tant que pile non vide:  
        noeud = pop(pile)  
        do_something_with_noeud(noeud)  
        pour chaque fils de enfants(noeud):  
            push(pile, noeud)
```

Exemple: Algorithme de parcours en largeur

```
function parcours_profondeur_it(noeud: Noeud):  
    file: FIFO = [ noeud ]  
  
    tant que file non vide:  
        noeud = pop(file)  
        do_something_with_noeud(noeud)  
        pour chaque fils de enfants(noeud):  
            push(file, noeud)
```

Graphes

- Un graphe est constitué de noeuds et de flèches reliant ces noeuds
- On peut considérer des graphes orienté ou non (si non orienté on parle d'arrêtes plutôt que flèches)
- On ajoute généralement de la donnée aux noeuds et flèches
 - L'exemple typique est un graphe représentant des lieux et dont les flèches sont étiquetés par une distance
- Un arbre est un cas particulier de graphe orienté et sans cycles (on parle de DAG)

Exemple: Algorithme de calcul de plus court chemin de Dijkstra

```
function dijkstra(graphe: Graphe, debut: Noeud) -> (Liste[Noeud], List[entiers])
    noeuds_traités: Liste[Noeud] = []

    predecesseurs = Liste[entiers]

    distances = List[entiers]
    distances[x] = infini pour tout noeud x
    distances[debut] = 0

    tant que noeuds(graphe) != noeuds_traités:
        choisir a dans noeuds(graphe) - noeuds_traités minimisant distances
        ajouter(a, noeuds_traités)
        pour chaque voisin b de a, si b n'est pas dans noeuds_traités:
            si distance[b] > distance[a] + poids(a, b):
                predecesseurs[b] = a
                distance[b] = distance[a] + poids(a, b)

    return predecesseurs, distances
```

Tables de hachage

- Structure d'indexation permettant d'associer de manière efficace des clefs à des valeurs
 - La structure conceptuelle associée est souvent appelée Dictionnaire
- Pour se faire, on utilise une fonction de hachage qui permet de transformer les clefs en nombre
 - Souvent une fonction utilisé aussi en cryptographie
 - On veut des bonnes propriétés: que deux clefs structurellement proche aient une valeur de hachage très éloigné
 - Que les clefs soit bien réparties dans l'espace des nombres
- Grace à la fonction de hachage, on peut distribuer les clefs dans des tableau de taille plus réduite, et donc plus rapide à parcourir
- Idéalement on veut un element par tableau, pour être en temps constant sur l'accès

Projet

Turingtoy

- Le sujet est dans le README du projet